

SECURITY IMPLEMENTATION REPORT

Strengthening Security Measures for a Web Application

Abstract

This report presents the implementation of essential security measures for a web application as part of Week 2 activities in the cybersecurity project titled "Strengthening Security Measures for a Web Application". The security enhancements focused on input validation and sanitization, password hashing, token-based authentication, and HTTP security header protection.

1. Project Objectives

- Implement secure coding practices
- Validate and sanitize user inputs
- Improve authentication security mechanisms
- Secure password storage using hashing techniques
- Implement token-based authentication
- Strengthen HTTP response security

2. Application Environment

Technology	Purpose
Node.js	JavaScript Runtime Environment
npm	Package Management
Express.js	Backend Framework
Angular	Frontend Framework
Validator.js	Input Validation
bcrypt	Password Hashing
JWT	Authentication
Helmet.js	HTTP Security Headers

3. Security Measures Implemented

3.1 Input Validation and Sanitization

Validator.js was implemented to validate and sanitize user inputs in order to prevent malicious or malformed data from entering the application.

```
const validator = require('validator');
```

```
if (!validator.isEmail(email)) {  
    return res.status(400).send('Invalid email');  
}
```

3.2 Password Hashing Using bcrypt

bcrypt was implemented to securely hash passwords before storing them in the database.

```
const bcrypt = require('bcrypt');  
  
const hashedPassword = await bcrypt.hash(password, 10);
```

3.3 Token-Based Authentication Using JWT

JSON Web Tokens (JWT) were implemented to improve authentication and secure session management.

```
const jwt = require('jsonwebtoken');  
  
const token = jwt.sign({ id: user._id }, 'your-secret-key');  
res.send({ token });
```

3.4 Securing HTTP Headers Using Helmet.js

Helmet.js was implemented to enhance browser-side security through secure HTTP headers.

```
const helmet = require('helmet');  
  
app.use(helmet());
```

4. Security Improvements Achieved

Security Measure	Security Improvement
Validator.js	Input validation and sanitization
bcrypt	Secure password storage
JWT	Secure authentication and session management
Helmet.js	Enhanced HTTP response security

5. Learning Outcomes

- Secure coding practices
- Input validation techniques
- Password hashing mechanisms
- JWT authentication systems
- HTTP security header configuration
- Defensive web application security techniques

6. Conclusion

This phase focused on implementing practical security mechanisms to strengthen the web application against common cyber threats. The implementation of validation libraries, password hashing, JWT authentication, and Helmet.js significantly improved the overall security posture of the application.

7. References

1. <https://owasp.org>
2. <https://github.com/juice-shop/juice-shop>
3. <https://github.com/validatorjs/validator.js>
4. <https://github.com/kelektiv/node.bcrypt.js>
5. <https://github.com/auth0/node-jsonwebtoken>
6. <https://helmetjs.github.io>

Author

Ashfaque Ali

Cybersecurity Project Research and Security Assessment